

Course Title: Node.js with MongoDB

1: Introduction to Node.js

Aim:

To study and understand the fundamentals of Node.js, its architecture, and its role in server-side development.

To create a basic HTTP server using Node.js and analyze how client-server communication works using request and response objects.

Objective:

- To understand Node.js runtime environment
- To learn how to create a basic web server
- To understand asynchronous execution

Procedure:

1. Install Node.js from the official website
2. Verify installation using `node -v`
3. Open VS Code and create a file named `server.js`

4. Import the built-in `http` module
5. Create a server using `createServer()` method
6. Handle request and response
7. Run the program using terminal

Program:

```
const http = require('http');

// Creating server
const server = http.createServer((req, res) => {

  // Setting response header
  res.writeHead(200, {'Content-Type': 'text/plain'});

  // Handling different routes
  if(req.url === '/'){
    res.write("Welcome to Node.js Server");
  }
  else if(req.url === '/about'){
    res.write("This is About Page");
  }
  else{
    res.write("Page Not Found");
  }

  // Ending response
  res.end();
```

```
});  
  
// Listening on port  
server.listen(3000, () => {  
  console.log("Server running at http://localhost:3000");  
});
```

Output:

- Browser displays different responses for `/`, `/about`, and other routes

Result:

Successfully created a Node.js server and handled client requests.

2: Express.js and Routing

Aim:

To study the Express.js framework and understand its advantages over the native HTTP module.

To implement routing in Express.js for handling multiple endpoints and simplify server-side development.

Objective:

- To understand Express framework
- To implement routing
- To simplify server creation

Procedure:

1. Initialize project using `npm init -y`
2. Install Express using `npm install express`
3. Create `app.js` file
4. Import Express module
5. Create application instance
6. Define routes
7. Run using `node app.js`

Program:

```
const express = require('express');  
const app = express();
```

```
// Home Route
```

```
app.get('/', (req, res) => {  
    res.send("Welcome to Express Home Page");  
});
```

```
// About Route
```

```
app.get('/about', (req, res) => {  
    res.send("About Page - Express Framework");  
});
```

```
});

// Contact Route
app.get('/contact', (req, res) => {
  res.send("Contact Page");
});

// Handling unknown routes
app.use((req, res) => {
  res.status(404).send("Page Not Found");
});

// Starting server
app.listen(3000, () => {
  console.log("Express server running on port 3000");
});
```

Output:

- Routes respond correctly for `/`, `/about`, `/contact`

Result:

Express routing implemented successfully.

3: Middleware in Express

Aim:

To understand middleware functions in Express.js and their role in processing HTTP requests.

To implement custom middleware for logging and request handling.

Objective:

- To understand middleware flow
- To implement logging middleware
- To control request lifecycle

Procedure:

1. Create Express server
2. Define middleware using `app.use()`
3. Log request details
4. Call `next()` function

Program:

```
const express = require('express');  
const app = express();
```

```
// Custom Middleware  
app.use((req, res, next) => {
```

```
    console.log("Request Method:", req.method);
    console.log("Request URL:", req.url);
    next();
  });

// Route
app.get('/', (req, res) => {
  res.send("Middleware Example");
});

app.listen(3000, () => {
  console.log("Server running...");
});
```

Output:

- Console displays request logs

Result:

Middleware successfully implemented.

4: REST API using Express (Without Database)

Aim:

To understand RESTful API design principles and implement CRUD operations using Express.js.

To simulate database operations using in-memory data structures.

Objective:

- To understand REST architecture
- To implement CRUD operations
- To handle JSON data

Procedure:

1. Create Express server
2. Use `express.json()` middleware
3. Create an array to store data
4. Implement GET, POST, PUT, DELETE

Program:

```
const express = require('express');
```

```
const app = express();
```

```
app.use(express.json());
```

```
// In-memory data
```

```
let students = [  
  {id:1, name:"John"},
```



```
    {id:2, name:"Sam"}  
  ];
```

```
// GET all  
app.get('/students', (req,res)=>{  
  res.json(students);  
});
```

```
// POST  
app.post('/students', (req,res)=>{  
  const newStudent = req.body;  
  students.push(newStudent);  
  res.json(newStudent);  
});
```

```
// PUT  
app.put('/students/:id', (req,res)=>{  
  const id = parseInt(req.params.id);  
  students = students.map(s => s.id === id ? req.body :  
s);  
  res.json({message:"Updated"});  
});
```

```
// DELETE  
app.delete('/students/:id', (req,res)=>{  
  const id = parseInt(req.params.id);  
  students = students.filter(s => s.id !== id);
```

```
res.json({message:"Deleted"});  
});  
  
app.listen(3000, ()=>console.log("Server running"));
```

Output:

- CRUD operations tested using Postman

Result:

REST API implemented successfully.

5: MongoDB Connection using Mongoose

Aim:

To understand MongoDB database and integrate it with Node.js using Mongoose.

To establish a connection between application and database.

Objective:

- To learn MongoDB basics

- To connect using Mongoose
- To handle connection errors

Procedure:

1. Install mongoose
2. Import mongoose
3. Use `connect()` method
4. Verify connection

Program:

```
const mongoose = require('mongoose');

// Connecting to MongoDB
mongoose.connect('mongodb://127.0.0.1:27017/studentD
B', {
  useNewUrlParser: true,
  useUnifiedTopology: true
})
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log("Connection Error:", err));
```

Output:

- Console shows connection success message

Result:

MongoDB connected successfully.

NODE.JS WITH MONGODB LAB MANUAL (WEEKLY PRACTICAL)

**Course Title: Node.js with MongoDB
Laboratory**

Duration: 2 Hours / Week

6: Schema and Model Creation

Aim:

To study and understand the concept of schemas and models in MongoDB using Mongoose.

To define the structure of documents and enforce data consistency within the database.

To create a model that acts as an interface for interacting with the database collection.

Objective:

- To understand schema design
- To create models
- To enforce structured data

Procedure:

1. Import mongoose
2. Define schema fields
3. Create model
4. Export model

Program:

```
const mongoose = require('mongoose');
```

```
const studentSchema = new mongoose.Schema({  
  name: {  
    type: String,  
    required: true  
  },  
  age: {  
    type: Number,  
    min: 18  
  }  
});
```

```
    },  
    department: String,  
    email: {  
      type: String,  
      unique: true  
    }  
  });  
  
const Student = mongoose.model("Student",  
studentSchema);  
  
module.exports = Student;
```

Output:

Schema and model created.

Result:

Successfully created schema and model.

7: Insert and Retrieve Data

Aim:

To understand how to insert documents into MongoDB and retrieve them using Mongoose methods.

To learn how to perform basic database operations like create and read.

Objective:

- To insert records
- To fetch records
- To understand async operations

Procedure:

1. Import model
2. Create new object
3. Save to database
4. Fetch using `find()`

Program:

```
const Student = require('./Student');
```

```
// Insert
```

```
const addStudent = async () => {  
  const s = new Student({  
    name: "John",  
    age: 20,  
    department: "CSE",  
    email: "john@gmail.com"  
  });
```

```
    await s.save();  
    console.log("Inserted");  
};  
  
// Fetch  
const getStudents = async () => {  
    const data = await Student.find();  
    console.log(data);  
};  
  
addStudent();  
getStudents();
```

Output:

Data inserted and displayed.

Result:

Insert and read operations successful.

8: Update and Delete Operations

Aim:

To understand how to update and delete documents in MongoDB using Mongoose methods.

To perform modification and removal of existing records.

Objective:

- To update records
- To delete records

Procedure:

1. Use `updateOne()`
2. Use `deleteOne()`

Program:

```
const Student = require('./Student');

// Update
Student.updateOne(
  { name: "John" },
  { age: 22 }
).then(() => console.log("Updated"));

// Delete
Student.deleteOne(
  { name: "John" }
).then(() => console.log("Deleted"));
```

Output:

Records updated and deleted.

Result:

Update and delete operations successful.

9: Validation and Error Handling

Aim:

To study validation techniques in Mongoose and implement error handling mechanisms.
To ensure only valid data is stored in the database.

Objective:

- To apply validation rules
- To handle errors

Program:

```
const mongoose = require('mongoose');
```

```
const schema = new mongoose.Schema({
  name: { type: String, required: true },
  email: {
    type: String,
    match: /.+\@.+\..+/
  }
});

const Model = mongoose.model("Test", schema);

const data = new Model({ name: "", email: "wrong" });

data.save()
.then()=>console.log("Saved"))
.catch(err=>console.log("Error:", err.message));
```

Result:

Validation and error handling implemented.

10: Express Router

Aim:

To understand modular routing in Express.js and organize application structure.

To separate route handling logic into different files for better maintainability.

Objective:

- To modularize routes
- To improve code structure

Program:

```
const express = require('express');  
const router = express.Router();
```

```
router.get('/', (req, res) => {  
    res.send("Student Route");  
});
```

```
module.exports = router;
```

Result:

Routes modularized successfully.

11: Query and Filtering

Aim:

To implement querying and filtering of data using MongoDB query operators.

To retrieve specific records based on conditions.

Objective:

- To filter data
- To use query parameters

Program:

```
app.get('/students', async (req, res) => {  
  const dept = req.query.department;  
  
  const data = await Student.find({  
    department: dept  
  });  
  
  res.json(data);  
});
```

Result:

Filtering implemented successfully.

12: Pagination

Aim:

To implement pagination in API responses to handle large datasets efficiently.

To control the number of records returned per request.

Objective:

- To implement limit and skip
- To improve performance

Program:

```
app.get('/students', async (req, res) => {  
  const page = parseInt(req.query.page) || 1;  
  const limit = 5;  
  
  const data = await Student.find()  
    .limit(limit)  
    .skip((page - 1) * limit);  
  
  res.json(data);  
});
```

Result:

Pagination implemented successfully.

13: Authentication using JWT

Aim:

To understand authentication mechanisms and implement JWT-based authentication.

To secure APIs and allow access only to authenticated users.

Objective:

- To generate JWT
- To verify JWT
- To protect routes

Program:

```
const jwt = require('jsonwebtoken');
```

```
const generateToken = () => {  
  const token = jwt.sign(  
    { userId: 1 },  
    "secret_key",
```

```
        { expiresIn: "1h" }  
    );  
  
    console.log(token);  
};  
  
generateToken();
```

Result:

JWT generated successfully.

14: Password Hashing using bcrypt

Aim:

To understand password security and implement hashing using bcrypt.

To securely store user passwords in the database.

Objective:

- To hash passwords
- To compare passwords

Program:

```
const bcrypt = require('bcrypt');

const hashPassword = async () => {
  const hashed = await bcrypt.hash("mypassword", 10);
  console.log("Hashed:", hashed);

  const match = await bcrypt.compare("mypassword",
hashed);
  console.log("Match:", match);
};

hashPassword();
```

Result:

Password hashing implemented successfully.

15: Mini Project - Student Management System

Aim:

To integrate all concepts learned throughout the course and develop a complete backend application.

To build a Student Management System with CRUD operations, authentication, and advanced features.

Objective:

- To build full backend system
- To integrate MongoDB
- To implement authentication

Features:

- User authentication (JWT)
- CRUD operations
- Pagination
- Filtering

Result:

Mini project implemented successfully.

COURSE OUTCOMES

- Develop server-side applications using Node.js
- Perform database operations using MongoDB
- Build REST APIs

- Implement authentication and security
-

INFERENCE

All experiments were performed successfully, and students gained practical knowledge in developing scalable backend applications using Node.js and MongoDB.
